

Scalable Agent Architecture Design

On Amazon Web Services

Layer10 x Nicolas el Khoury



Nicolas El Khoury
Engineering Excellence Expert
Software - Infrastructure - DevOps



I. Executive Summary

I. Executive Summary

- The question this document addresses is not how to use artificial intelligence. It is how to build infrastructure around it that is reliable enough to trust with real work, secure enough to operate across multiple customers without leaking information between them, and scalable enough to grow without being redesigned.
- The system described here is an autonomous agent platform built exclusively on AWS. A user submits a goal through a React frontend. The platform accepts it, executes it through a stateless orchestration layer that calls Claude via AWS Bedrock, and delivers the result back to the browser through a persistent WebSocket connection, without polling and without manual intervention.
- Every action the agent takes is recorded to a durable event log before it is executed. If any component fails at any point during execution, a new instance reads the log and resumes from the last recorded event. Nothing is lost. The agent does not hold credentials. It cannot reach external services directly.



I. Executive Summary

- Every authenticated call to an external system passes through a dedicated proxy that retrieves the appropriate customer credential from AWS Secrets Manager, makes the call, and returns only the result.
- A compromised execution environment yields access to an isolated working directory and nothing else. The security properties of the system are structural, not advisory. They hold because the architecture makes violations physically impossible, not merely prohibited.
- The decisions recorded in this document were made in response to specific constraints, not preferences. Every technology choice has an alternative that was considered and a reason it was not selected. Several decisions remain open, documented explicitly so that the boundaries of what has been resolved are clear. The system is designed to accommodate those resolutions without requiring architectural changes when they are made. What is built today is intended to function under conditions that are not yet fully known. That requirement shaped every decision in this document.

II. Problem Statement

A. Deceptive Systems

- The most common way to build an AI agent system is to place everything in one process. The orchestration logic, the execution environment, the session state, the credentials, all of it runs together on a single machine. In development, this functions. In a controlled demonstration, it is convincing. The output appears, the task completes, and there is little visible reason to examine what sits beneath the surface.
- This is precisely the condition under which structural weakness develops most reliably. Not when a system is clearly failing, but when it appears to be working.
- The problems described in this section are not visible at the moment they are created. Each one forms quietly, through decisions that are individually reasonable and collectively dangerous. Their effects accumulate beneath the appearance of function, and they become observable only when conditions require the system to operate beyond the boundaries it has been quietly developing from the beginning.

B. Stateful Processes

- An AI agent does not execute a single transaction. It works through a sequence of decisions that may span dozens of steps, involve multiple external interactions, and consume significant time before reaching a conclusion. Each step produces a record: what the model decided, which action it took, what the result was, what followed. In the naive approach, this record exists in one place only. The memory of a running process.
- The system continues to function as long as nothing interrupts that process. When interruption occurs, and across a system running concurrent sessions over extended periods, interruption is not an exceptional event but a statistical certainty, the accumulated record disappears with it. There is no checkpoint, and no basis for resumption. The only available response is to begin again from zero.

B. Stateful Processes

- This does not present itself as a vulnerability during the period in which it is being created. Sessions complete, work is delivered, and the surface remains intact. The structural condition becomes visible only at the moment of failure, when the full cost of holding state exclusively in memory reveals itself at once, in full.
- The consequence is not addressed through operational discipline or more careful process management. It is rather addressed through architecture. State that exists only inside a running process will be lost when that process ends unexpectedly. The response is to remove state from the process entirely: to maintain a durable, append-only event log outside the process that records every meaningful event before the process acts on it. When a process ends, nothing is lost. A new process reads the log, reconstructs the state exactly as it was, and resumes from the last recorded point. The log becomes the source of truth. The process becomes disposable.

C. Keys and Secrets Management

- An AI agent interacts with the world. It reads and writes files, executes commands, calls external services, and operates against real infrastructure. In the naive approach, all of this execution occurs inside the same process that holds the credentials required to do it: API keys, service tokens, database passwords, access credentials of every kind.
- This creates a structural condition that no policy or access control can resolve. The attack surface is the model itself.
- Large language models can be manipulated through the content they process. Text embedded in a file the agent reads, in a webpage it fetches, in a response it receives from an external service, can be constructed to influence the model's behavior in ways its operator never intended. This technique, known as prompt injection, does not require access to the system's infrastructure. It requires only that malicious content reach the model's context.

C. Keys and Secrets Management

- In the naive architecture, a successful manipulation does not compromise a single operation. It compromises everything the process can reach. The malicious instruction executes in the same environment that holds every credential available to the agent. No additional breach is required. The attacker needs only to influence what the model decides.
- This cannot be resolved by greater care in what the model is allowed to read. The nature of language models is that any content entering the context is potential input to the decision-making process. The structural resolution is to ensure that credentials are physically unreachable from the environment where execution occurs. Not restricted by policy. Not protected by access controls that a sufficiently clever instruction might circumvent. Unreachable by design, because they were never placed there.
- The architectural response separates the execution environment from the orchestration layer and introduces a dedicated proxy between execution and any external service. The execution environment holds no credentials under any circumstance. Every call to an external service passes through the proxy, which retrieves the appropriate credential from a secure vault, makes the authenticated request, and returns only the result. A fully compromised execution environment yields nothing of strategic value, because nothing of strategic value was ever placed inside it.



D. Scalability

- A system in which every concern is coupled to every other concern has one scaling path: vertical. More capacity on the same machine. This path is expensive, carries its own ceiling, and requires downtime to execute. More fundamentally, it addresses none of the conditions that make the workload difficult to manage in the first place.
- AI agent workloads do not arrive predictably and do not consume resources uniformly. Demand is uneven across time. Individual tasks vary significantly in duration and complexity. Some complete in seconds. Others run for extended periods. Some are compute-intensive. Others spend the majority of their time waiting for responses from external systems. A single machine running all of this simultaneously will be either chronically over-provisioned or chronically insufficient, and the decision between those two conditions must be made before demand is known.
- Beyond capacity, there is a dependency problem. When orchestration, execution, and storage are coupled within a single process, no component can be scaled, updated, or replaced without affecting the others. A deployment that requires restarting the process terminates every active session simultaneously. A performance problem in one component degrades every other component sharing the same resources. A change to the execution environment requires taking the orchestration layer offline. The system moves as a unit because it was built as a unit, and that property does not change through operational adjustment.

D. Scalability

- The resolution is decomposition. Each concern is separated into an independently deployable component with a single responsibility and a well-defined interface. The orchestration layer runs as stateless workers. Because they hold no state of their own, they can be scaled to any number of instances without coordination between them.
- The execution environment runs as a pool of isolated containers, scaled according to its own demand signal, independent of the orchestration layer. A queue between them absorbs uneven load without losing work and without requiring either layer to be aware of the other's capacity. Each component can be scaled, updated, restarted, and replaced on its own terms.
- The system that results handles one concurrent session and one thousand with the same architecture. The difference is the number of instances, not the design.

E. Scope of Work

- The session store addresses the fragility of state held in memory. No work is lost because no work is held only in memory. Every event is written to a durable log before the system acts on it. Any component can fail at any moment and the system resumes without manual intervention.
- The sandbox and credential proxy address the structural vulnerability of execution environments that hold credentials. The two are separated by design. A compromised execution environment is a contained event. It does not become a platform breach, because there was nothing in the execution environment worth taking.
- The decomposed, stateless, queue-driven architecture addresses the scaling ceiling of coupled systems. Each layer responds to its own demand. The system does not move as a unit, because it was not built as one.

These are not improvements to the naive approach. They are its replacement.

III. Architecture Overview

A. Structure

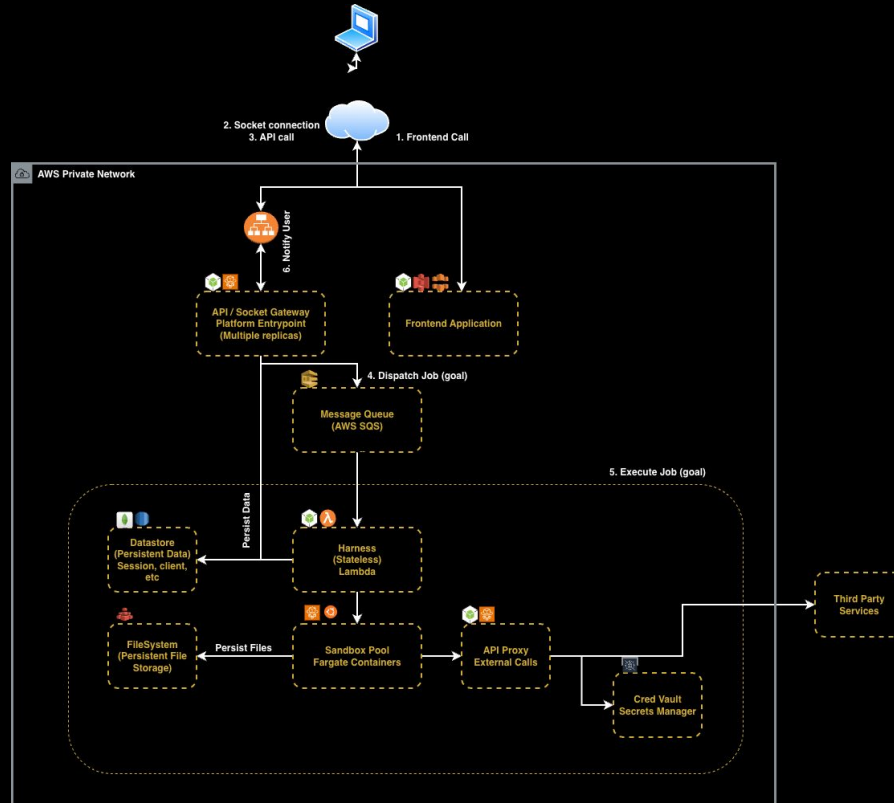
- The system described in this document assumes deployment on AWS exclusively. Every infrastructure decision, service selection, and scaling mechanism referenced in the sections that follow is specific to the AWS ecosystem. The architecture does not abstract over cloud providers and does not attempt to remain portable. It is designed to use AWS primitives directly, at the level where they provide the most control and the clearest operational guarantees.
- The system is composed of independently deployable components arranged in layers, each with a single responsibility and a well-defined interface to the components adjacent to it. No layer has visibility into the internal operation of another. Each communicates only through the contract it exposes: a request goes in, a result comes out. What happens in between is the concern of that component alone.

A. Structure

- This property is not incidental. It is the condition that makes the system resilient, scalable, and auditable. When a component fails, the failure is contained within its boundary. When a component is replaced, the rest of the system continues without modification. When the behavior of one layer needs to change, the change does not propagate outward.
- The sections that follow describe this structure in two dimensions. The infrastructure dimension covers what runs, where it runs, and how traffic moves between components at the network level. The software architecture dimension covers what each component does, how it is designed internally, and what principles govern its interaction with the rest of the system.



A. Structure



B. Infrastructure Architecture

- The entry point for all traffic is an Application Load Balancer operating at Layer 7. It receives every request directed at the platform and routes it based on the host header. HTTP traffic arriving is routed to the gateway service, which handles REST requests. Socket traffic arriving is routed to the gateway service, which handles persistent WebSocket connections. The host-based routing decision is made at the load balancer level, before any application code executes.
- The gateway service runs as multiple replicas behind the load balancer. WebSocket connections are persistent by nature: once a browser establishes a connection to a specific replica, all subsequent communication on that connection must flow through the same replica.
- To maintain this property across multiple instances, the ALB is configured with sticky sessions using duration-based cookies. When a browser first connects, the ALB assigns it a cookie that binds it to a specific replica for the duration of the session. All subsequent requests from that browser, including the WebSocket connection and any REST calls made during the same session, are routed to the same replica. If a replica fails, the client reconnects and the ALB assigns a new binding. The interruption is momentary and the session resumes from the last recorded state in the datastore.

B. Infrastructure Architecture

- The frontend is served entirely outside this path. The React application is built into static assets and stored in an S3 bucket. CloudFront sits in front of that bucket, handling global distribution, SSL termination, and cache management. A request never reaches the load balancer. It is resolved at the CloudFront edge and served from S3 directly. The frontend and the API are structurally independent, sharing only the domain and the browser that connects to both.
- Behind the gateway, all components operate in isolation from the public internet. The message queue is AWS SQS, which receives job submissions from the gateway and holds them until a harness worker is available to process them. The harness workers are AWS Lambda functions (or Fargate containers for long-running tasks), invoked by SQS directly. Lambda handles concurrency natively: when multiple messages are present in the queue simultaneously, multiple Lambda instances are invoked in parallel without any additional configuration. Each invocation is independent and shares no state with any other.



B. Infrastructure Architecture

- The sandbox pool runs on ECS Fargate. Each container is provisioned from a minimal image built on Alpine Linux, runs as a non-root user, and is scoped to a single working directory for the duration of its session. Where the output of an agent session requires persistence beyond the container lifecycle, an EFS volume is mounted into the container at the working directory path. The container terminates. The files remain.
- The credential vault is AWS Secrets Manager. Every customer's credentials are stored as a separate secret, keyed by customer identifier. No other component in the system holds credentials directly. The datastore is a document database serving as the session event log, the customer registry, and the WebSocket connection registry. It is the only component with durable knowledge of what has occurred across the system.



C. Software Architecture

- The gateway is a single Node.js (or Python) process running two servers. An Express HTTP server handles REST routes: job submission, session status, and customer management. A WebSocket server manages persistent browser connections. When a browser opens a WebSocket connection, the gateway records the connection identifier alongside the session identifier in the datastore. This record is the mechanism by which the completion notification reaches the correct client.
- When a harness worker finishes a session, it calls an internal endpoint on the gateway replica that holds the matching connection. The gateway reads the connection identifier from the datastore, confirms the connection is local to this replica, and pushes the result directly to the connected browser. The browser receives the notification without polling and without any knowledge of the internal execution path that produced it.

C. Software Architecture

- The harness worker is a stateless function. It holds no data of its own between the moment it is invoked and the moment it exits. When SQS delivers a message, the worker reads the session identifier, retrieves the full event log from the datastore, and reconstructs the agent state entirely from that record.
- It then enters the agent loop: it calls the Claude API with the reconstructed context, receives a response, writes the response to the event log before acting on it, and either executes tool calls or terminates depending on the stop condition. Every write to the event log occurs before the corresponding action is taken. If the worker is terminated at any point during execution, the event log reflects the last completed state. A new invocation reads the same log and resumes from that point. The worker is disposable. The log is not.

C. Software Architecture

- The sandbox is an isolated execution environment accessible to the harness through a single HTTP endpoint: a tool name and input go in, a string result comes out. The sandbox has no knowledge of the harness, no access to the datastore, and no credentials of any kind. It executes the instructions it receives within a scoped filesystem and returns the result. When a tool call requires interaction with an external service, the sandbox does not make that call. It forwards the request to the API proxy.
- The API proxy is a HTTP process running on Fargate that sits between the sandbox and the external world. It receives a tool call request accompanied by a session identifier. It uses the session identifier to look up the customer identifier in the datastore, then uses the customer identifier to retrieve the appropriate credentials from Secrets Manager. It makes the authenticated call to the external service and returns only the result. The sandbox receives a string. It never receives, handles, or sees a credential.



D. Component Interactions

- The system has no circular dependencies and no shared mutable state between components. Each component reads from one source, writes to one destination, and exposes one interface. The datastore is the single point through which asynchronous coordination occurs: the gateway writes session identifiers and WebSocket connection identifiers, the harness writes and reads event logs, and the proxy reads customer identifiers. No component communicates with another directly except through its defined interface.
- This structure has a practical consequence for failure. When any component fails, the failure propagates no further than its interface boundary. A sandbox that crashes mid-execution produces a tool call error, which the harness writes to the event log as a result and passes to Claude for handling. A gateway replica that restarts loses its in-memory WebSocket connections, which the client re-establishes on reconnect through the ALB sticky session mechanism. A harness invocation that times out leaves the session log in its last recorded state, from which any subsequent invocation can resume. In each case, the failure is visible, bounded, and recoverable without manual intervention.
- The system does not tolerate the failure of the datastore or the credential vault, as both are dependencies of multiple components. Both are managed services with their own availability guarantees and replication mechanisms, and both are treated as infrastructure rather than application components. Their failure modes are addressed in the resilience section.



IV. Request and Execution Flow



A. Frontend Call

- The user opens the React application. The request never reaches the load balancer. CloudFront resolves it at the edge and serves the static assets directly from S3. The browser renders the interface, and the user submits a task.
- This submission is a REST call, which the ALB receives and routes to an available gateway replica based on the host header. The ALB assigns a duration-based sticky session cookie to the browser at this point, binding it to the replica that received the request. All subsequent communication from this browser travels to the same replica for the duration of the session.



B. Socket connection

- Before or immediately after submitting the task, the browser opens a persistent WebSocket connection. The ALB routes this connection to the same gateway replica identified by the sticky session cookie. The gateway accepts the connection, generates a connection identifier, and writes it to the datastore alongside the session identifier that was created when the task was submitted.
- This record is the only mechanism the system uses to route the completion notification back to the correct browser. No other component is aware of the WebSocket connection.



C. API Call

- The REST request carrying the task goal arrives at the gateway. The gateway creates a session record in the datastore containing the session identifier, the customer identifier derived from the authenticated request, the goal, and an initial status of pending.
- It returns the session identifier to the browser immediately. The browser does not wait for the task to complete. It holds the session identifier and the open WebSocket connection. When the result is ready, it will arrive through that connection.



D. Dispatch Job

- The gateway writes a message to AWS SQS containing the session identifier and the customer identifier. This is the handoff between the synchronous entry layer and the asynchronous execution layer. Once the message is in the queue, the gateway's responsibility for this session is limited to delivering the completion notification when the harness signals it.
- SQS holds the message durably until a harness worker is available to process it. If demand is high and all workers are occupied, the message waits in the queue without being lost and without blocking any other session.

E. Execute Job

- **SQS delivers the message to a harness worker. The worker reads the session identifier, retrieves the full event log from the datastore, and reconstructs the agent state from that record. For a new session, the event log contains only the initial session record and the user goal. The worker enters the agent loop.**
- **On each iteration, the worker calls the Claude API with the reconstructed context. Claude reads the full event history and determines the next action. If Claude produces a final response, the worker writes the session complete event to the datastore and proceeds to step 6. If Claude requests a tool call, the worker writes the tool call event to the datastore before executing it, then routes the request to the sandbox.**



E. Execute Job

- The sandbox receives the tool name and input through its HTTP endpoint and executes the instruction within its isolated filesystem. If the tool call requires an external service, the sandbox forwards the request to the API proxy rather than making the call directly. The proxy receives the request alongside the session identifier, looks up the customer identifier in the datastore, retrieves the appropriate credentials from Secrets Manager, makes the authenticated call to the external service, and returns only the result as a plain string. The sandbox passes the result back to the harness. The harness writes the result to the datastore before continuing. The loop repeats.
- This continues until Claude produces a final response or a safety condition terminates the session. At every point in this loop, the event log in the datastore reflects the last completed state. A worker that is terminated at any point during execution leaves the session in a recoverable condition. A new invocation reads the same log and resumes from the last recorded event.

F. Notify User

- When the harness worker writes the session complete event, it calls an internal endpoint on the gateway. The gateway reads the connection identifier associated with the session from the datastore, confirms the WebSocket connection is local to this replica, and pushes the result directly to the browser through the open connection.
- The browser receives the result without polling, without refreshing, and without any visibility into the execution path that produced it. The session is complete. The event log contains a full and ordered record of every decision, every tool call, every result, and every transition that occurred from the moment the task was submitted to the moment the result was delivered.

V. Component Breakdown



A. Frontend App

- The frontend is a React application. It is the only surface the end user directly interacts with. It is responsible for rendering the interface, authenticating the user, submitting task goals to the gateway, opening and maintaining the WebSocket connection, and displaying results as they arrive.
- The application is built into static assets at deployment time and stored in an S3 bucket. It has no server-side runtime. CloudFront sits in front of the bucket and serves the assets globally from edge locations, handling SSL termination and cache invalidation on each deployment. A request resolves at the CloudFront edge and never reaches any internal AWS infrastructure.
- This deployment model has a direct consequence for security. There is no application server to compromise, no server-side session to hijack, and no backend process running under the frontend's identity. The application is inert until the browser executes it. All sensitive operations occur in the backend components that the frontend calls.

B. Application Load Balancer

- The ALB is the single entry point for all traffic directed at the platform's backend services. It operates at Layer 7 and makes routing decisions based on the host header of each incoming request. HTTP traffic arriving is routed to the gateway's REST endpoint. Socket traffic arriving is routed to the gateway's WebSocket endpoint. No backend component is reachable from the public internet through any other path.
- Sticky sessions are enabled on the ALB using duration-based cookies. When a browser first connects, the ALB assigns it a cookie that binds it to a specific gateway replica. All subsequent requests from that browser, both REST and WebSocket, travel to the same replica for the duration of the session. This property is required for WebSocket connections, which are stateful at the transport layer and cannot be transferred between replicas without terminating and re-establishing the connection.
- The ALB performs health checks against each gateway replica continuously. When a replica fails its health check, the ALB removes it from rotation. Browsers bound to the failed replica will reconnect, at which point the ALB assigns them to a healthy replica and issues a new sticky session binding. The interruption is limited to the reconnection interval. Session state is recovered from the datastore.

C. API and WebSocket Gateway

- The gateway is a HTTP process running two servers within the same operating system. An HTTP server handles REST routes: task submission, session status queries, and customer management. A WebSocket server manages persistent browser connections. Both servers share the same process memory, which allows the WebSocket server to push notifications directly when the harness signals completion, without any inter-process communication.
- The gateway runs as multiple replicas behind the ALB. Each replica is stateless with respect to business logic. A replica that receives a task submission creates the session record in the datastore and writes the job to SQS. It does not retain any local record of the session beyond the WebSocket connection identifier, which it holds in memory for the duration of the connection.
- When a harness worker completes a session, it calls an internal endpoint on the gateway. Because the ALB sticky session mechanism ensures the harness always reaches the replica holding the relevant WebSocket connection, the gateway can locate the connection in memory, confirm it is active, and push the result directly to the browser. If the connection has been lost since the session began, the gateway records the completion in the datastore and the browser retrieves the result when it reconnects and polls for session status.

D. AWS SQS Message Queue

- SQS is the boundary between the synchronous entry layer and the asynchronous execution layer. The gateway writes one message per session to the queue. The message contains the session identifier and the customer identifier. Nothing else is required, because the harness worker derives all remaining context from the datastore using those two values.
- SQS provides two properties that are critical to the architecture. The first is durability. A message written to SQS is not lost if the consumer fails before processing it. The message becomes visible again after the visibility timeout expires and is redelivered to another worker. The second is decoupling. The gateway does not need to know how many harness workers are available or whether any are currently occupied. It writes to the queue and returns. The queue absorbs the gap between submission rate and processing capacity without either layer being aware of the other's state.
- Messages that fail processing repeatedly are moved to a dead letter queue. This provides an inspection point for sessions that could not be executed, without losing the original message or affecting the processing of other sessions.

E. Harness Workers

- The harness worker is a stateless execution unit. It is invoked by SQS, runs the agent loop for a single session, and exits. It holds no state between invocations. Every piece of information it needs to execute is retrieved from the datastore at the start of each invocation, and every piece of information it produces is written to the datastore before the next action is taken.
- The agent loop operates as follows. The worker retrieves the session event log from the datastore and reconstructs the full conversation context from that record. It calls the Claude API with the reconstructed context and receives a response. If the response is a final answer, the worker writes the session complete event to the datastore and signals the gateway. If the response is a tool call request, the worker writes the tool call event to the datastore, routes the request to the sandbox, receives the result, writes the result to the datastore, and repeats the loop.

E. Harness Workers

- The write-first principle governs every step. An event is written to the datastore before the action it describes is taken. If the worker is terminated between the write and the action, the event log reflects a state from which any subsequent invocation can resume. The worker is cattle. The event log is not.
- Concurrency is handled natively by the underlying runtime. When SQS contains multiple messages simultaneously, multiple worker instances are invoked in parallel. Each instance is independent and operates against its own session. No coordination between instances is required because no instance holds state that another instance needs.

F. Sandbox Pool

- The sandbox is an isolated execution environment. It receives tool call requests from the harness through a single HTTP endpoint, executes them within a scoped filesystem, and returns a string result. It has no knowledge of the harness, no connection to the datastore, and no credentials of any kind.
- Each sandbox runs as an ECS Fargate container provisioned from a minimal Alpine Linux image with a Node.js or Python runtime. The container runs as a non-root user. Its filesystem is scoped to a single working directory for the duration of the session it serves. Where the session produces file output that must survive beyond the container lifecycle, an EFS volume is mounted into the container at the working directory path. When the container terminates, the EFS directory persists and remains accessible for retrieval or inspection.
- The security property the sandbox provides is structural. Generated code executes inside it. Credentials do not. A sandbox that is compromised through prompt injection or malicious tool output yields access to an isolated filesystem and nothing else. It cannot reach the datastore, cannot call external services directly, and cannot access any credential. Every external call is routed through the proxy, which operates outside the sandbox boundary.

G. API Proxy

- The API proxy is an HTTP service running on ECS Fargate. It sits between the sandbox and any external service the agent needs to reach. Its single responsibility is to receive unauthenticated tool call requests from the sandbox, attach the appropriate customer credentials, make the authenticated call to the external service, and return the result.
- When the proxy receives a request, it extracts the session identifier and uses it to look up the customer identifier in the datastore. It checks its in-memory credential cache for an entry keyed to that customer identifier. If a valid cached entry exists, it uses it. If not, it retrieves the credential from Secrets Manager, caches it with a time-to-live, and proceeds. It makes the authenticated call to the external service and returns only the result as a plain string. The sandbox receives a string. It never receives, handles, or sees a credential at any point in this process.
- The proxy runs as a long-lived Fargate service rather than a serverless function. This allows it to maintain persistent connections to external services, cache credentials in memory between requests from the same customer, and serve repeated calls with consistent low latency. It is not exposed to the public internet. Only sandbox containers can reach it, enforced through VPC security group rules.

H. Persistent Filesystem

- The filesystem layer provides durable storage for file output produced during agent execution. Its purpose is narrow: to preserve files that the agent creates when those output must survive beyond the container lifecycle. The appropriate storage mechanism depends on how the session uses those files. Two options are available and selected at session configuration time based on the access pattern the task requires.
- The first is EFS. An EFS volume is mounted into the sandbox container at the working directory path, making it indistinguishable from local disk from the perspective of the running process. The agent reads and writes files using standard filesystem operations without any code change or API call. EFS is the correct choice when the agent needs to read files it has previously written within the same session, when a task involves iterative file modification across multiple tool calls, or when the output must remain accessible for retrieval or inspection after the container terminates. The EFS volume is mounted per session at a path scoped to the session identifier. Two sessions running simultaneously do not share filesystem access. When the container terminates, the session directory remains on the EFS volume until a scheduled cleanup process removes it according to a configured retention policy.

H. Persistent Filesystem

- The second is S3. When a session produces file output that is written once and retrieved asynchronously after the session completes, S3 is the more appropriate choice. The sandbox writes the output file to S3 at the end of execution through an explicit API call. S3 provides lower cost per gigabyte than EFS, no throughput ceiling under normal conditions, and native integration with downstream processes that may need to consume the file. It is the correct choice for report generation, asset export, and any task where the agent does not need to re-read what it has written during the same session.
- Not every session requires either. Sessions whose output is a text result returned through the event log require only the container's local filesystem. EFS and S3 are provisioned selectively. The selection is made at session initialization and passed to the sandbox as a configuration parameter so the execution environment is prepared before the agent loop begins.

I. Credentials Vault

- The credentials vault is AWS Secrets Manager. Every customer's credentials are stored as an individual secret, keyed by customer identifier. No other component in the system stores credentials. The harness does not hold them. The sandbox does not hold them. The gateway does not hold them. Only the proxy retrieves them, and only at the moment a tool call requires it.
- Secrets Manager provides automatic rotation for credentials that support it. When a credential is rotated, the proxy's cache entry for that customer expires at its configured time-to-live and the next request retrieves the updated value. No component restart is required. No deployment is needed.
- Access to Secrets Manager is controlled through IAM roles attached to the proxy's ECS task definition. No other component has an IAM policy that permits Secrets Manager access. The permission is structural, not advisory.

J. Datastore

- The datastore is a document or relational database that serves as the central coordination point for the system. It holds three categories of data: the session event log, the customer registry, and the WebSocket connection registry.
- The session event log is an append-only sequence of documents, one per event, ordered by timestamp. It records every meaningful occurrence in the lifecycle of a session: the initial goal, each Claude response, each tool call, each tool result, and the final completion or error event. This log is the source of truth. Every harness invocation derives its execution context entirely from this record. No information about what occurred in a session exists anywhere else in the system in a durable form.

J. Datastore

- The customer registry holds the mapping between customer identifiers and the metadata required to serve them: their account status, their plan, and the reference key used to retrieve their credentials from Secrets Manager. The proxy reads from this registry on each tool call. The gateway reads from it on each authenticated request.
- The WebSocket connection registry maps session identifiers to gateway connection identifiers. The gateway writes to it when a browser opens a WebSocket connection. The harness reads from it when signaling completion. This record is ephemeral in the sense that a connection identifier becomes invalid when the browser disconnects, yet durable in the sense that it survives gateway replica restarts and is used to recover the notification path when a replica is replaced.



VI. Security Model



The security properties of this architecture are structural. They do not depend on configuration, policy enforcement, or the correct behavior of any individual component. They hold because the system is designed so that the violations they prevent are physically impossible, not merely prohibited. A misconfigured component, a compromised process, or a manipulated model cannot circumvent them by finding a different path, because the path does not exist.



A. Credential Isolation

- **No credential ever enters the sandbox. This is not a rule that is checked at runtime. It is a consequence of how the system is assembled. The sandbox is provisioned without credentials. No environment variable carries a key. No mounted volume contains a token. No IAM role attached to the sandbox task definition grants access to Secrets Manager or to any external service. The only thing the sandbox can do when it needs to call an external service is forward the request to the proxy through the internal HTTP interface it was given at startup. It has no alternative path.**
- **The proxy holds credentials. The sandbox does not. These two components are separated by a network boundary enforced through VPC security group rules. The sandbox can reach the proxy. It cannot reach Secrets Manager, cannot reach external services directly, and cannot reach the datastore. The security group configuration is the enforcement mechanism. It does not rely on the sandbox behaving correctly.**
- **The consequence of this separation is that prompt injection, the technique by which malicious content in the agent's context manipulates the model into taking unintended actions, cannot produce credential exfiltration. A successful injection that causes the sandbox to execute arbitrary code yields access to the sandbox filesystem. Nothing else is reachable from that position. The attacker has gained an isolated working directory. The credentials, the customer data, and the session history remain outside the boundary that was breached.**



B. Customer Isolation

- Every customer's credentials are stored as a separate secret in Secrets Manager, keyed by customer identifier. The proxy retrieves a credential only after confirming which customer owns the session that generated the request. A session belonging to Customer A cannot produce a credential belonging to Customer B, because the lookup path runs through the session record in the datastore, which was written by the gateway at the moment of job submission using the authenticated identity of the requesting user.
- The session record is the boundary. A harness worker operating on a session cannot access any data outside that session. It reads the event log for its own session identifier and nothing else. It does not have query access to other sessions, other customers, or other event logs. The datastore access pattern is scoped by session identifier at the application layer, and IAM policies restrict the harness to the operations it requires.
- The sandbox is further isolated by the working directory scoping. Each container is assigned a directory path that includes the session identifier. A sandbox serving one session cannot traverse into a directory belonging to another session. The EFS mount point is shared infrastructure, but the access path within it is unique per session and not discoverable by another container without knowledge of the session identifier it was not given.

C. Execution Isolation

- **Generated code runs inside the sandbox. The sandbox is a container running as a non-root user on ECS Fargate. Its filesystem access is limited to the working directory it was assigned. Its network access is limited to the proxy endpoint it was given at startup. It has no outbound internet access, no access to the AWS metadata service, and no access to any other internal component.**
- **The non-root execution constraint removes the class of attacks that depend on elevated process privileges. A process running as root inside a container has capabilities that a non-root process does not: it can modify system files, bind to privileged ports, and in certain configurations interact with the container runtime in ways that affect the host. Running as a non-root user with no additional capabilities removes these vectors without requiring any runtime enforcement.**
- **The network boundary removes the class of attacks that depend on the sandbox reaching something it should not. Even if generated code attempts to call an external service, query the AWS metadata endpoint, or connect to the datastore, the security group rules discard the packets before they arrive. The attempt produces a connection timeout. Nothing is returned. No information is leaked.**

C. Execution Isolation

- Fargate provides the container boundary itself. The sandbox process runs inside a container that is managed by AWS and isolated from other containers running on the same underlying infrastructure. The isolation is provided at the infrastructure level and does not depend on the application running inside the container.
- These layers operate independently. The loss of any one of them does not remove the others. A process that escapes the non-root constraint still faces the network boundary. A process that finds a network path still faces the absence of credentials. Each layer reduces the surface available to the next.

D. Audit Trail

- The session event log is append-only. Events are written to it before the actions they describe are taken. Nothing is deleted from it during the normal operation of the system. Every decision the model made, every tool call it requested, every result it received, and every transition between states is recorded in the order it occurred, with a timestamp, for every session that has ever run. This record serves two purposes that are distinct in practice but unified in the structure that produces them.
- The first is operational. The event log is how the harness reconstructs state after a failure and resumes from the last recorded point. Its append-only property is an operational requirement, not a security feature. Yet the same property that makes it useful for recovery makes it useful for audit: a log that cannot be modified in the course of normal operation is a log whose contents can be trusted.
- The second is accountability. When a question arises about what an agent did, the event log is the answer. It does not require inference, reconstruction from side effects, or correlation across multiple systems. The record is direct and ordered. Every action the agent took is present in the log that governed its execution.



D. Audit Trail

- Access to the event log is controlled through IAM policies attached to each component's task or function role. The harness can read and write the logs for its own sessions. The gateway can read session status. The proxy can read customer identifiers. No component has write access to another component's records, and no component has delete access to any session event.
- The combination of append-only writes, pre-action recording, and access-controlled reads produces an audit trail that reflects what actually occurred rather than what was reported after the fact. It does not depend on any component choosing to record its actions honestly. The structure of the system requires the record to be written before the action is possible.

VII. Scalability Model



*Each component in this architecture scales independently. No component's capacity is coupled to another's.
This section describes the scaling mechanism, the scaling trigger, and the practical ceiling for each component.*



A. Gateway

- The gateway runs as an ECS service with a target tracking auto-scaling policy. The scaling metric is CPU utilization or active WebSocket connection count, whichever reaches its threshold first. When the metric exceeds the configured target, ECS provisions additional task replicas and the ALB begins distributing new connections across them. Existing sticky session bindings are unaffected. Only new connections are distributed to the new replicas.
- The practical ceiling is determined by the ALB connection limit and the VPC subnet capacity, both of which are configurable and well above the thresholds at which application-level performance becomes a concern. The gateway does not hold session business logic, so the cost of adding a replica is the cost of starting a container and nothing more.
- Scaling down follows the same policy in reverse. When utilization drops below the lower threshold, ECS drains connections from excess replicas before terminating them. Active WebSocket connections are given time to complete or disconnect naturally during the drain period.

B. AWS SQS

- **SQS does not scale in the conventional sense. It has no capacity to configure and no instances to provision. It accepts messages at any rate the gateway can produce them and holds them until a consumer is available. The queue depth grows when production outpaces consumption and shrinks when consumption catches up. At no point does a high queue depth affect the gateway's ability to write or the harness worker's ability to read.**
- **The queue depth is the primary observable metric for this layer. A consistently growing queue depth indicates that the harness worker pool is undersized relative to submission rate. A consistently empty queue indicates excess worker capacity. Queue depth drives the scaling decision for the harness layer, not for SQS itself.**



C. Harness Workers

- The harness worker is a Lambda function triggered directly by SQS. Lambda manages concurrency natively. When multiple messages are present in the queue simultaneously, Lambda invokes a separate function instance per message without any additional configuration. When the queue empties, idle instances are released automatically. There is no scaling policy to define, no instance pool to manage, and no coordination required between concurrent invocations.
- Each invocation is fully independent. Concurrent workers reading and writing to the datastore simultaneously do not interfere with one another because access is scoped by session identifier at the application layer. Adding concurrent invocations does not affect sessions already in progress.



C. Harness Workers

- The reserved concurrency limit on the Lambda function is the primary scaling control. It sets the maximum number of simultaneous agent sessions the harness layer will execute at any given moment and acts as a rate-limiting mechanism against the Claude API. As concurrent invocations increase, the aggregate rate of Claude API calls increases with them. At sufficient scale, the Claude API rate limit, measured in requests per minute and tokens per minute per account, becomes the binding constraint. This ceiling is managed through Anthropic's API tier system and is addressable through account-level limit increases.
- The Lambda execution timeout is the other relevant constraint. Lambda enforces a maximum execution duration of fifteen minutes per invocation. Agent sessions that are expected to run longer than this limit require the harness to be deployed as an ECS Fargate service instead, which carries no execution time ceiling. This decision is addressed in the Technology Decisions section.

D. Sandbox Pool

- The sandbox pool scales on the number of active harness sessions that have initiated tool calls. Each harness worker that enters a tool call iteration requires an available sandbox. When no sandbox is available, the worker waits. If wait times exceed acceptable thresholds, additional sandbox containers are provisioned.
- Sandbox scaling is managed through ECS service auto-scaling using a custom metric derived from active tool call request rate. The metric is emitted by the proxy, which receives every tool call request and can observe the rate at which requests arrive and the latency at which they are served. A rising latency trend triggers scale-out. A falling request rate triggers scale-in after a stabilization period.

D. Sandbox Pool

- The sandbox pool scales independently of the harness pool. The ratio of sandboxes to harness workers is not fixed. A workload with frequent, short tool calls requires more sandboxes per worker. A workload with infrequent tool calls or long-running ones requires fewer. The pools adjust to the workload profile without any architectural change.
- The practical ceiling for the sandbox layer is EFS throughput when file-intensive sessions are running concurrently. EFS throughput scales with storage consumed in bursting mode and is configurable through provisioned throughput mode when burst capacity is insufficient.

E. API Proxy

- The proxy scales on request rate and response latency using ECS service auto-scaling. Because the proxy maintains an in-memory credential cache, scaling out introduces a cold cache effect on new replicas: their first request for a given customer credential will miss the cache and hit Secrets Manager. Subsequent requests will hit the cache. This effect is transient and bounded by the Secrets Manager request rate limit, which is generous relative to the credential retrieval pattern the proxy produces.
- The proxy does not need to scale proportionally to the sandbox pool. A single proxy instance can serve multiple concurrent sandbox requests. The ratio depends on the latency of the external services being called, which determines how long each proxy thread is occupied per request.

F. Datastore

- The datastore scales on read and write throughput. The session event log is append-only, which means write operations are inserts and read operations are ordered sequential reads by session identifier. This access pattern is well-suited to document databases, which handle high insert rates and keyed sequential reads efficiently without requiring complex query planning.
- Read replicas absorb the read load from gateway status queries, harness context reconstruction, and proxy customer lookups. The primary instance handles only writes. This separation allows the read layer to scale horizontally while the write layer remains a single authoritative source.
- The practical ceiling for the datastore is determined by the write throughput of the primary instance. In practice, the write rate is bounded by the number of active harness sessions and the iteration rate of each session's agent loop, both of which grow slowly relative to the capacity of a managed database primary.



G. Scaling Summary

Component	Scaling mechanism	Scaling trigger	Practical ceiling
Gateway	ECS target tracking	CPU or WebSocket connection count	ALB connection limit
SQS	Native, no configuration	Not applicable	No practical ceiling
Harness workers	Lambda native concurrency	SQS message delivery	Claude API rate limit and 15-minute execution timeout
Sandbox pool	ECS custom metric scaling	Tool call request rate and latency	EFS throughput
API proxy	ECS target tracking	Request rate and response latency	Secrets Manager request rate
Datastore	Read replicas and primary scaling	Read and write throughput	Primary write throughput



VIII. Resilience and Failure Handling



Resilience in this architecture is not a feature added on top of the system. It is a consequence of the same structural decisions that govern security and scalability. Components that hold no state cannot lose state when they fail. Logs that are written before actions are taken can be replayed when the process that read them no longer exists. Boundaries that prevent components from reaching one another also prevent failures from propagating between them. This section describes what happens when each component fails, what the system recovers automatically, and what requires human intervention.

A. Gateway Failure

- When a gateway replica fails its ALB health check, the ALB removes it from rotation immediately. No new connections are routed to it. Browsers that were bound to the failed replica through the sticky session mechanism will attempt to reconnect. The ALB assigns them to a healthy replica and issues a new binding.
- The reconnecting browser loses its in-memory WebSocket connection. It does not lose its session. The session identifier is held by the browser and the session record exists in the datastore. The browser reconnects, re-establishes the WebSocket connection with the new replica, and the new replica writes the updated connection identifier to the datastore.
- If the session was already complete at the moment of reconnection, the browser retrieves the result through a status poll. If the session is still executing, the browser waits on the new WebSocket connection and receives the completion notification when the harness signals it.



A. Gateway Failure

- The harness worker that was executing the session is unaffected by the gateway failure. It continues running the agent loop and signals completion by calling the gateway endpoint when the session finishes. The gateway reads the current connection identifier from the datastore, which now points to the new replica's connection, and pushes the result to the browser.
- No session data is lost. No intervention is required. The failure is visible to the browser as a momentary disconnection and nothing more.

B. SQS Failure

- **SQS is a managed service with an availability commitment from AWS. Its failure mode in practice is not outage but delayed delivery, which the architecture tolerates without any special handling. Messages written to the queue are durable. They are not lost if SQS experiences a transient disruption. They are delivered when the disruption resolves.**
- **If a harness worker fails while processing a message, the message becomes visible again in the queue after the visibility timeout expires. The default visibility timeout should be set to exceed the maximum expected session duration to prevent a session from being delivered while it is still being processed. When the worker fails, the session event log reflects the last completed state. The delivered message triggers a new invocation, which reads the log and resumes from that point.**
- **Messages that fail processing repeatedly, beyond the configured maximum receive count, are moved to the dead letter queue. The dead letter queue provides an inspection point for sessions that could not be executed after multiple attempts. Each entry retains the original message, allowing the failure to be diagnosed and the session to be manually resubmitted if appropriate.**

C. Harness Worker Failure

- A Lambda invocation that fails mid-execution leaves no orphaned state. The event log in the datastore reflects every event that was written before the failure occurred, and nothing that was not. The write-first principle ensures that no action was taken without a corresponding log entry, and no log entry was written for an action that did not complete.
- When the SQS message becomes visible again after the visibility timeout, Lambda invokes a new instance. The new instance reads the event log, reconstructs the agent context from the last recorded state, and resumes the session. From Claude's perspective, the conversation history is intact. From the datastore's perspective, the session is continuous. The failure is invisible to the end result.
- The one failure mode that requires attention is a worker that times out at the Lambda execution ceiling of fifteen minutes. If a session's agent loop has not completed within this window, the invocation is terminated and the message is redelivered. If the session consistently exceeds the timeout, it indicates either a runaway loop that the safety guards failed to catch, or a genuinely long-horizon task that requires the harness to run on ECS Fargate rather than Lambda. Both conditions are detectable through CloudWatch duration metrics and require intervention to resolve.

D. Sandbox Failure

- A sandbox container that fails mid-execution produces a failed HTTP response to the harness worker that called it. The harness writes the failure as a tool call error event to the datastore and passes the error to Claude as the tool result. Claude receives a string indicating the tool call failed and determines the next action based on that information. It may retry the tool call, attempt an alternative approach, or surface the error in its final response depending on the nature of the failure and the instructions it was given.
- The sandbox failure is contained at the tool call boundary. It does not terminate the session, does not affect the event log beyond the error entry, and does not require the harness to restart. The session continues from the point of failure with Claude deciding how to proceed.
- If the sandbox pool becomes exhausted and no container is available to serve a tool call, the harness worker waits. The wait introduces latency into the session but does not cause a failure. ECS provisions additional sandbox containers in response to the demand signal, and the tool call is served when a container becomes available. If the wait exceeds the Lambda execution timeout, the session is redelivered and resumes from the last recorded event, which predates the failed tool call.

E. API Proxy Failure

- A proxy failure produces a failed HTTP response to the sandbox that called it. The sandbox returns the failure to the harness as a tool result string. The harness writes it to the datastore and passes it to Claude. The session continues.
- Because the proxy runs as a long-lived ECS Fargate service with multiple replicas behind an internal load balancer, the failure of a single replica does not affect sessions in progress on other replicas. The internal load balancer routes the next request to a healthy replica. The credential cache on the failed replica is lost, which means the first request to the new replica for each affected customer will miss the cache and retrieve the credential from Secrets Manager. This is a latency event, not a failure.
- If the entire proxy service becomes unavailable, all tool calls that require external services will fail for the duration of the outage. Sessions that do not require external tool calls are unaffected. Sessions that do will accumulate tool call errors in their event logs and Claude will respond accordingly. When the proxy recovers, the sessions resume normally.



F. Persistent Filesystem Failure - EFS

- EFS is a managed service with a multi-availability-zone replication architecture. Its failure mode in practice is elevated latency rather than data loss or complete unavailability. Sessions that are writing to EFS during a period of elevated latency will experience slower tool call responses but will not lose data.
- If EFS becomes completely unavailable during an active session, file write operations will fail and return errors to the harness as tool results. The session event log captures these failures. Sessions that do not require file persistence and use only the container's local filesystem are unaffected entirely.
- Data written to EFS before a failure is not at risk. EFS replicates data across availability zones synchronously. A failure in one zone does not affect data written in another.



F. Persistent Filesystem Failure - S3

- S3 is a managed service with eleven nines of durability and a multi-availability-zone storage architecture. Its failure mode in practice is elevated API latency or intermittent request errors rather than data loss. Sessions configured to write output to S3 will experience failed write operations during a disruption, which the harness records as tool results in the session event log.
- If the S3 write at the end of a session fails, the session event log retains the full record of what the agent produced. Depending on the nature of the output, the harness can be configured to retry the write on the next invocation using the event log as the source. Data successfully written to S3 before a failure is not at risk. S3 stores each object redundantly across a minimum of three availability zones and does not expose single-zone failure as a data loss event.
- Sessions using only the container's local filesystem are unaffected by failures in either EFS or S3.



G. Datastore Failure

- The datastore is the single component whose failure affects the entire system simultaneously. Every other component depends on it for reads or writes during normal operation. Its failure mode is therefore the most consequential and the one that requires the most careful management.
- The datastore runs with a primary instance and one or more read replicas across availability zones. If a read replica fails, read traffic is rerouted to the remaining replicas or to the primary. Gateway status queries and proxy customer lookups experience increased latency but continue to function. No session data is affected.
- If the primary instance fails, write operations cease. The harness cannot write event log entries, which means the write-first principle cannot be satisfied, which means the agent loop cannot safely proceed. Running harness invocations will stall at the next write attempt. Lambda invocations will time out and be redelivered once the primary recovers or a replica is promoted. SQS holds all pending messages durably during the outage period.

G. Datastore Failure

- **Managed database services on AWS provide automated failover through Multi-AZ deployment. When the primary fails, a replica is promoted to primary automatically, typically within sixty to one hundred and twenty seconds. Harness invocations that stalled during this window will be redelivered and resume from the last successfully written event. No session data written before the failure is lost.**
- **The datastore and the credential vault are the two components for which failure requires reliance on the managed service's own resilience mechanisms rather than the application architecture's. Both are configured for multi-availability-zone deployment and both carry AWS service level commitments. Their failure handling is an infrastructure concern rather than an application concern.**



F. Failure Handling Summary

Component	Automatic recovery	Requires intervention
Gateway replica	ALB reroutes traffic, client reconnects, session resumes from datastore	None
SQS	Messages redelivered after visibility timeout	Dead letter queue inspection for repeated failures
Harness worker	New Lambda invocation resumes from event log	Sessions exceeding 15-minute timeout
Sandbox container	Tool call error passed to Claude, session continues	None
API proxy replica	Internal load balancer reroutes to healthy replica	Full proxy outage affecting external tool calls
Persistent filesystem	Elevated latency, no data loss	Complete EFS unavailability affecting file operations
Datastore read replica	Traffic rerouted to remaining replicas	None
Datastore primary	Automated Multi-AZ failover within 60 to 120 seconds	Extended outage beyond managed failover window

